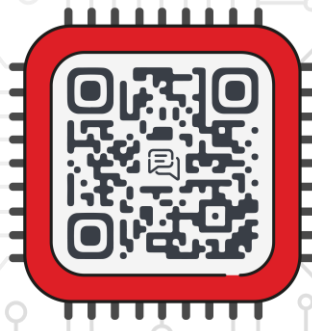




8



14



2100 sp

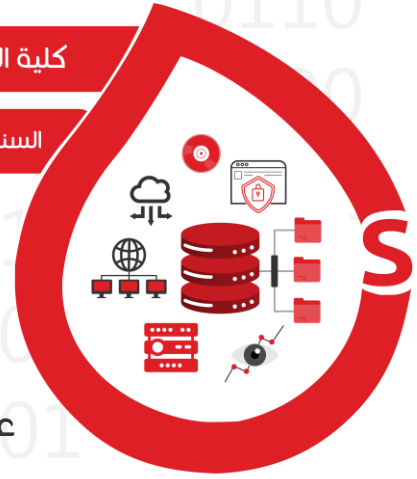
عملي

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Procedure and Function

عملي مشترك



RB Informatcs; 15/12/2025

محتوى مجاني غير مخصص للبيع التجاري

قواعد معطيات ٢

سنتعرف في هذه المحاضرة على مفهوم ال Procedure الممتع والموفر للوقت.

■ عند كتابة أي استعلام (Query)، يقوم المُحسِّن (Optimizer) بخطوتين قبل تنفيذ الفعلي (Execute):

1. الخطوة الأولى هي ترجمة (Compile) الاستعلام للتأكد من صحته (النحوية وكذلك من وجود ال Data).

2. الخطوة الثانية هي إنشاء خطة التنفيذ (Execution Plan) (المناسبة).

نلاحظ أن الزمن اللازم لهاتين الخطوتين اللتين يقوم بهما المُحسِّن قليل جداً (جزء من الثانية)، لكن عند تنفيذ عدد هائل من الاستعلامات، فإن هذا الزمن يتراكم ليصبح دقائق، وهو أمر غير مقبول. ومن هنا، ظهرت أهمية الإجراءات المخزنة (Stored Procedures) والدوال (Functions).

■ إذاً، ما هي الإجراءات المخزنة (Stored Procedures)؟

هي عبارة عن كود SQL مُجهز يمكن حفظه في قاعدة البيانات لإعادة استخدامه مراراً وتكراراً. عندما يكون لدينا استعلام نستخدمه بشكل متكرر، نقوم بحفظه كإجراء مخزن (Procedure)، ثم نستدعي هذا الإجراء لتنفيذه. في المرة الأولى التي يتم فيها استدعاء الإجراء، يقوم المُحسِّن بترجمته وإنشاء خطة التنفيذ (Execution Plan) الخاصة به، ثم يقوم بتخزين هذه الخطة في ذاكرة التخزين المؤقت (Cache Memory). وفي كل مرة يتم استدعاؤه لاحقاً، يقوم SQL Server باستخدام خطة التنفيذ المخزنة مباشرة من الذاكرة، متجاوزاً بذلك خطوتي الترجمة وإنشاء الخطة.

ملاحظة:

■ يقوم المُحسِّن بتخزين خطة التنفيذ (Execution Plan) في ذاكرة التخزين المؤقت (Cache Memory)، ولكنه لا يخزن بيانات الجداول (Data) أو نتائج الاستعلام (Results) نفسها.

ولكن، قد تظهر لهذه العملية بعض السلبيات في حالات نادرة جداً، وتعرف هذه المشكلة بـ "Parameter Sniffing":

■ عند أول استدعاء للإجراء، إذا كانت قيم (Parameters) الممررة تؤدي إلى استعلام صغير، فإن المُحسِّن ينشئ خطة تنفيذ مُحسَّنة لهذا الحجم الصغير.

■ ولكن، إذا تم استدعاء الإجراء مرة أخرى بقيم معلمات مختلفة تؤدي إلى استعلام ضخم (يعالج ملايين السجلات)، فسيقوم SQL Server بإعادة استخدام خطة التنفيذ الأولى (السيئة) المخزنة في الذاكرة، مما يؤدي إلى أداء ضعيف جداً.

إذاً، الهدف الأساسي من استخدام الإجراءات المخزنة (Procedures) والدوال (Functions) هو تحسين الأداء (Performance) وتقليل الزمن اللازم لتنفيذ الاستعلامات المتكررة.

يُنصح بشدة بعدم كتابة استعلامات SQL مباشرة (Direct Queries) داخل كود التطبيق (Application)، بل يتم استدعاء الإجراءات المخزنة (Stored Procedures) الموجودة في قاعدة البيانات.

عند كتابة procedure على مستوى الـ DB كاملة، فإن أي تعديل على الـ query يكون ضمن الـ procedure، أما إذا تمت الكتابة كـ Backend بدون procedure، ف عند أي تعديل سأضطر لتوليد version جديد، لأن ظهور أي خطأ لن يضطرنا لتعديله إلا مرة واحدة، لذلك ينصح بوضع الـ logic ضمن الـ DB. أما غير ذلك فسنضطر لتعديله من أكثر من مكان وسيكون ذلك صعب جداً.

■ كيفية انشاء ال Procedure؟

إنشاء Stored Procedure

```
CREATE PROCEDURE PROCEDURE NAME
AS
SQL STATEMENT
```

تنفيذ Stored Procedure

```
EXEC procedure_name
```

مثال عملي لتطبيق المفاهيم :

لنقم بإنشاء الجدولين :

```
CREATE TABLE publishers)
pub_id VARCHAR(10) PRIMARY KEY,
pub_name VARCHAR(100)
city VARCHAR(50)
country VARCHAR(50)
);
```



التعب الذي تصرب منه اليوم

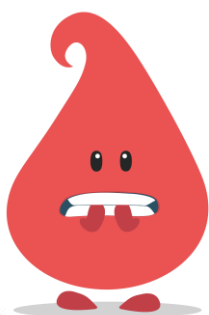
ستدفع ثمنه غداً مضاعفاً

```
CREATE TABLE titles(  
    title_id VARCHAR (10) PRIMARY KEY,  
    title VARCHAR(150)  
    type VARCHAR(30)  
    price DECIMAL(6,2)  
    pubdate DATE,  
    pub_id VARCHAR(10)  
    FOREIGN KEY (pub_id) REFERENCES publishers(pub_id)  
);
```

ولنقم الان بانشاء مايلي :

```
CREATE PROCEDURE spGetBookData  
AS  
BEGIN  
    SELECT title_id, type, title, price  
    FROM titles;  
END;  
  
EXECUTE spGetBookData;  
DROP PROCEDURE spGetBookData;
```

عند التنفيذ سيعيد ال RESULT SET التي تتضمن المعلومات المطلوبة:
عند القيام بمعالجة البيانات داخل ال Procedure، لن نقوم بكتابة SQL مجردة، بل سنقوم بكتابة بلغة تتناسب مع ال Compiler، وهذه اللغات تختلف باختلاف ال Engine، وفي ال SQL Server هي ال T/SQL.



ليس كل صمت حكمة

أحياناً هو خوف متخفٍ

```
CREATE PROCEDURE spGetBookPubData
AS
BEGIN
    SELECT country, city, pub_name
    FROM publishers
    ORDER BY country, city;

    SELECT title, type, price, pub_name
    FROM publishers p
    INNER JOIN titles t ON t.pub_id = p.pub_id
    ORDER BY pub_name;
END;
EXECUTE spGetBookPubData;
DROP PROCEDURE spGetBookPubData;
```

■ الشرح:

1. الإجراء الأول: spGetBookData (مرحلة التأسيس)

هذا الجزء يعلمك "القواعد الأساسية" لكتابة الإجراء:

الهيكل: يبدأ بـ CREATE وينتهي بـ END.

الوظيفة: هو مجرد استعلام بسيط لجلب بيانات الكتب.

دورة الحياة: (أنشأناه ← جربناه بـ Execute ← حذفناه بـ DROP). هذه هي الطريقة التي يتدرب بها المبرمج لتجربة الكود وتعديله.

2. الإجراء الثاني: spGetBookPubData (مرحلة الاحتراف)

هنا تظهر القوة الحقيقية:

داخل هذا الإجراء، طلبنا من قاعدة البيانات تنفيذ استعلامين منفصلين (واحد للناشرين وواحد للكتب). في البرمجة العادية، ستحتاج لإرسال طلبين من التطبيق إلى السيرفر، لكن هنا بطلب واحد فقط (Execute) تحصل على كل النتائج.

■ شرح ال keywords في المثال:

- تبدأ العملية بأمر CREATE PROCEDURE، وهو الأمر المسؤول عن حجز اسم خاص لهذا الكود داخل ذاكرة قاعدة البيانات، مما يغنيها عن إعادة كتابة الأوامر الطويلة في كل مرة. بعد ذلك تأتي كلمة AS التي تعمل كفصل منطقي، حيث تخبر القاعدة بأن ما سيأتي بعدها هو "متن" أو "محتوى" هذا الإجراء.

- لتحسين هذا المحتوى، نستخدم الكلمتين BEGIN و END، وهما بمثابة "الصور" أو "الحاوية" التي تضم جميع أوامر SQL بداخلها؛ فكل ما يُكتب بينهما يُعامل ككتلة واحدة تُنفذ معًا بالتسلسل. وفي قلب هذه الحاوية، نستخدم SELECT لتحديد البيانات المطلوبة، و FROM لتحديد الجداول التي سنستخرج منها تلك البيانات.
- وعندما نريد تنظيم النتائج بشكل احترافي، نستخدم خاصية ORDER BY التي تقوم بترتيب البيانات (مثل ترتيب الناشرين حسب الدولة والمدينة) بدلاً من ظهورها بشكل عشوائي. وأخيرًا، يأتي دور كلمة EXECUTE التي تعتبر "زر التشغيل" الفعلي؛ فعند كتابتها بجانب اسم الإجراء، يتم استدعاء وتنفيذ كل تلك الأوامر المخزنة دفعة واحدة وبسرعة عالية.

■ الخلاصة:

هذه الكلمات تعمل معًا لتحويل مجموعة من الأوامر المشتتة إلى "برنامج صغير مخزن"، تبدأ بخلقه وتسميته، ثم تحدد حدوده وأوامره، وفي النهاية تناديه بكلمة واحدة ليعطيك النتائج كاملة.

أي أنه سيكون لدينا هنا N RESULT SET وليس واحدة فقط.

أول RESULT SET ستكون قيمة أول QUERY وهي ال (COUNTRY, CITY, PUB_NAME).

ثاني RESULT SET سيكون قيمة ثاني QUERY وهي ال JOIN.

وهنا نلاحظ أنه حتى ال PROCEDURE يدعم أكثر من QUERY لنتمكن من الاستفادة منه بأكثر من RESULT

SET.

مثال 2:

```
CREATE PROCEDURE spGetBookDataByType
    @Type VARCHAR(20)
AS
BEGIN
    SELECT title, pubdate, pub_name
    FROM publishers p
    INNER JOIN titles t ON t.pub_id = p.pub_id
    WHERE type = @Type;
END;
EXECUTE spGetBookDataByType 'business';

DROP PROCEDURE spGetBookDataByType;
```

■ الشرح:

- في المثال السابق، كان الإجراء يعطيك "كل شيء". أما هنا، الإجراء spGetBookDataByType مصمم ليعطيك الكتب التابعة لنوع معين فقط (مثل كتب التجارة 'business') يحدده المستخدم عند التشغيل.
- كما استخدم الكود عملية الربط (INNER JOIN) لجمع بيانات من جدولين مختلفين (الناشرين والكتب) في نتيجة واحدة.
- تبدأ العملية بتعريف الإجراء عبر CREATE PROCEDURE، ولكن الجديد هنا هو إضافة @ Type VARCHAR(20)؛ هذا يسمى "بارامتر"، وهو بمثابة "فراغ" أو "خانة" نتركها للمستخدم ليدخل فيها النوع الذي يريد البحث عنه. كلمة AS تتبعها BEGIN و END لتحديد النطاق البرمجي الذي يحتوي على صلب العملية.
- داخل الإجراء، نجد جملة SELECT التي تطلب أعمدة محددة، لكن القوة تكمن في INNER JOIN؛ هذه التعليمة تعمل كجسر يربط جدول الناشرين (publishers) بجدول الكتب (titles) باستخدام مفتاح مشترك هو pub_id. لكي لا تظهر جميع الكتب، نستخدم type = @Type، وهي شرط يخبر القاعدة: "أظهر فقط الكتب التي يطابق نوعها القيمة التي أدخلها المستخدم في البداية".
- أخيراً، عند استخدام EXECUTE، نضع بجانب اسم الإجراء القيمة المطلوبة (مثل 'business'). هذه القيمة تذهب مباشرة لتسكن مكان @Type وتنفذ الاستعلام بناءً عليها، مما يجعل الكود مرناً جداً بدلاً من كتابة كود منفصل لكل نوع كتاب.

مثال 3:

```
CREATE PROCEDURE spgetbooktitlebyid
    @id VARCHAR(6),
    @bname VARCHAR(100) OUTPUT
AS
BEGIN
    SELECT @bname = title
    FROM titles
    WHERE title_id = @id;
END;

DECLARE @name VARCHAR(100);
EXECUTE spgetbooktitlebyid 'BU1111', @name OUTPUT;
PRINT 'title is: ' + @name;
```

■ الشرح:

- تبدأ العملية باستخدام CREATE PROCEDURE لتعريف الإجراء، وهنا نلاحظ وجود نوعين من البارامترات: الأول هو @id لاستقبال رقم الكتاب، والثاني هو @bname المتبوع بكلمة OUTPUT. هذه الكلمة هي "مفتاح السر"، فهي تخبر قاعدة البيانات أن هذا المتغير ليس مجرد معلومة تدخل للإجراء، بل هو "وعاء" سيخرج منه معلومة بعد انتهاء التنفيذ.
- داخل كتلة الـ BEGIN والـ END، لا نقوم بعرض البيانات مباشرة، بل نستخدم جملة = @bname SELECT. هذه الحركة تسمى "التخصيص" (Assignment)، حيث يتم أخذ اسم الكتاب من الجدول وحفظه داخل المتغير @bname.
- أما عند التشغيل، نستخدم أولاً DECLARE لخلق متغير خارجي (مثل @name) ليكون هو المستلم للنتيجة. ثم نستخدم EXECUTE مع ذكر كلمة OUTPUT مرة أخرى بجانب المتغير لضمان انتقال القيمة من داخل الإجراء إلى خارجه. وأخيراً، نستخدم أمر PRINT لعرض النتيجة النهائية المدمجة مع نص توضيحي.

ملاحظة:

الفائدة البرمجية: هذا النوع من الإجراءات لا يُستخدم لعرض البيانات للمستخدم فحسب، بل يُستخدم داخل الأنظمة البرمجية لنقل البيانات بين العمليات المختلفة.

■ ما هو الاختلاف عن المثال السابق:

الإجراء السابق يطبع النتيجة ويرحل، أما هذا الإجراء فهو "يسلمك النتيجة في يدك" (المتغير) لتفعل بها ما تشاء. ونلاحظ أنه هنا لن يعيد Result Set، بل سيعيد قيمة معينة عبر الـ OUTPUT parameter، بالإضافة إلى رسالة نصية عبر أمر PRINT.

Function

مفهوم الـ Function في قاعدة البيانات لا يختلف عن مفهوم الـ Function الذي مر معنا في البرمجة، فهي تأخذ argument وترجع value.

■ الفرق بين الـ Function والـ Procedure:

1. Function:

- ترجع قيمة واحدة.
- يمكن استخدامها داخل استعلام (Query).
- يمكن تضمينها في جملة SELECT.

لا أحد قادم لإنقاذك

إما أن تنهض أو تبقى



2. Procedure:

- يمكن أن يُرجع قيمة (عبر OUTPUT)، أو مجموعة نتائج (Result Set)، أو كليهما.
- لا يمكن استخدامه داخل استعلام.
- لا يمكن استخدام جملة SELECT لاستدعاء إجراء مباشرة.

مثال عملي:

```
CREATE FUNCTION fn_Get_Pub_Name_ById(@Id VARCHAR(10))
RETURNS VARCHAR(100)
AS
BEGIN
    RETURN (SELECT pub_name
            FROM publishers
            WHERE pub_id = @Id);
END;
```

هنا في هذا المثال، سوف تقوم الدالة بإرجاع اسم دار النشر على أساس الـ ID الذي نمرره لها. وسيتم استخدامها ضمن استعلام SELECT بالشكل التالي:

مثال لاستدعاء الدالة بقيمة مباشرة --

```
SELECT dbo.fn_Get_Pub_Name_ById('0736') AS pub_name;
```

مثال لاستخدام الدالة مع جدول --

```
SELECT title, dbo.fn_Get_Pub_Name_ById(pub_id) AS pub_name
FROM titles;
```

في حال قمنا بتمرير ID غير موجود، فإنها تعيد القيمة NULL:

```
SELECT dbo.fn_Get_Pub_Name_ById('7') AS pub_name;
```

الشرح:

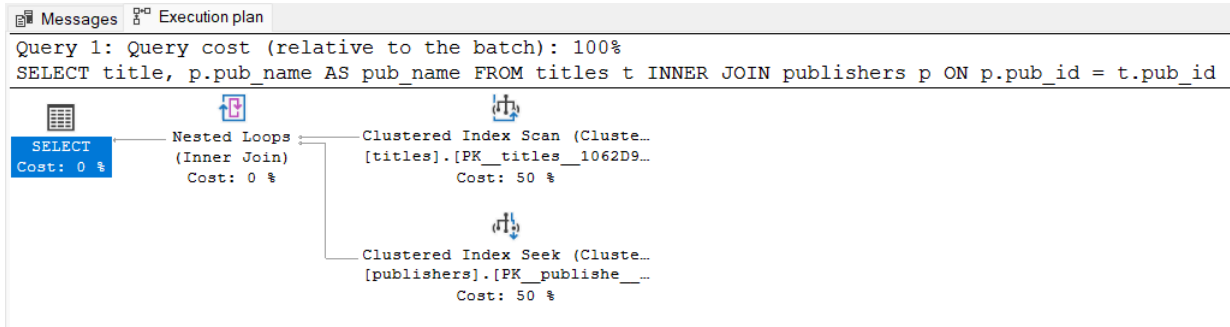
- هنا نقوم بطلب جلب الـ title، ثم نستدعي الدالة ونمرر لها القيمة pub_id من جدول titles لكل صف، ليتم جلب اسم دار النشر إلى جانب العنوان.
- ما هي الفائدة منها بهذه الطريقة؟

توفر علينا كتابة INNER JOIN أو sub query في كل مرة.

لنقم بتنفيذها بالطريقتين الآخرين ومقارنة الأداء:

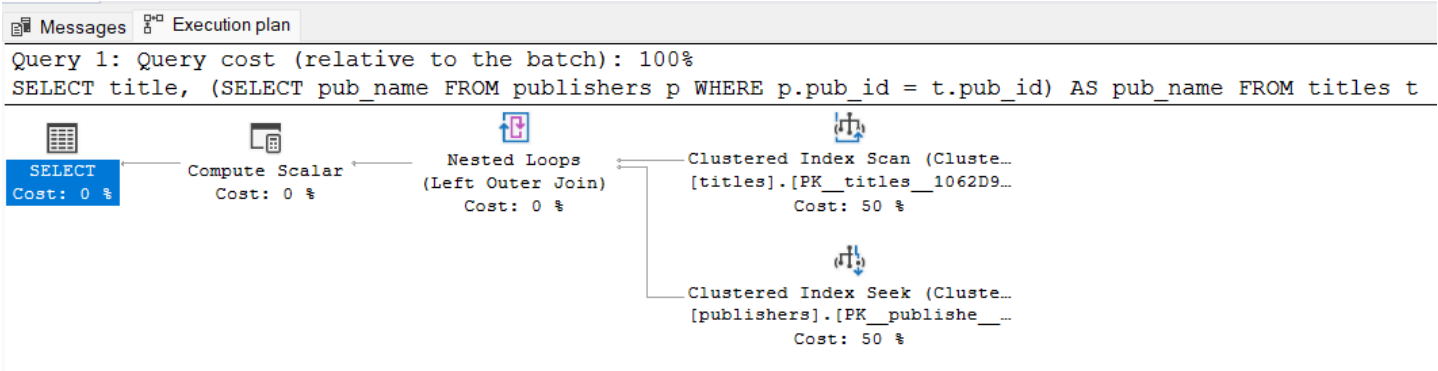
1. طريقة 1 INNER JOIN :

```
SELECT title, p.pub_name AS pub_name
FROM titles t
INNER JOIN publishers p ON p.pub_id = t.pub_id;
```



2. طريقة 2 Sub Query :

```
SELECT title, (SELECT pub_name
FROM publishers p
WHERE p.pub_id = t.pub_id) AS pub_name
FROM titles t;
```



نلاحظ أن الطرق الثلاث أعطت نفس النتيجة. لكن أيها أفضل؟

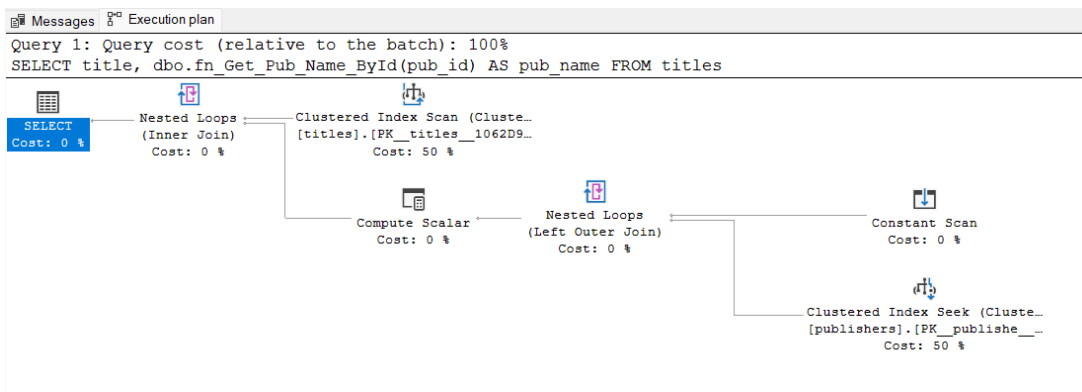
تحليل خطة التنفيذ:

- في الـ INNER JOIN نلاحظ أن المُحسَّن قام بمسح جدول titles مرة واحدة (Clustered Index Scan) وجلب كل الصفوف. بالتزامن، قام بعمل Seek على جدول publishers لكل صف تم جلبه من titles. هذه هي آلية تنفيذ الـ Nested Loops لجلب pub_name عن طريق pub_id.
- في الـ Sub Query نلاحظ أن خطة التنفيذ كانت مطابقة تماماً لطريقة INNER JOIN، لأن المُحسَّن الذكي يتعامل مع الاستعلام الفرعي المرتبط (Correlated Subquery) بنفس الطريقة.

أما بالنسبة لخطة التنفيذ الخاصة بالدالة (Function):

```
SELECT title, dbo.fn_Get_Pub_Name_ById(pub_id) AS pub_name
FROM titles;
```

- أولا قام بعمل scan لجداول ال titles وتم التنفيذ لمرة واحدة وجلب ال row 18 وعمل loop مع ال result set القادمة من تنفيذ ال Fun والتي تم تنفيذها لمرة واحدة بينما تم تنفيذ ال function ل 18 مرة ومن ثم قام بعمل ال seek لل الحصول على ال result set التي تم تنفيذها لمرة واحدة.
 - إذا، كيف نعرف أي الطرق أفضل؟
- نقوم بعمل إحصائيات من خطط التنفيذ (Actual Execution Plan) ونقارن تكلفة كل عملية (Query Cost).



نلاحظ انه لل الfunction: تم عمل scan. لجداول ال titles لمرة واحدة وعمل logical reads (2) ونلاحظ انه في ال sub query:

قام بعمل scan لجداول ال publishers. لمرة واحدة وعمل logical reads (37) وبالإضافة ل scan لجداول ال titles. لمرة واحدة وال logical reads (2)

اما في ال inner join قام بعمل ال scan. ل جداول ال titles. لمرة واحدة مع ال logical reads (2) اذا افضل اداء هو اداء ال function او ال inner join.

ولكنها ليست قاعدة وملاحظ انه باختلاف ال data تختلف طريقة الاداء الافضل ولكن التوابع تكون الافضل بنسبة 90٪ وهنا يكمن دور ال tuning اي التجربة في الحالات الخاصة لمعرفة اي الطرق افضل في اداء تلك الحالة وال result set تلعب دورا كبيرا في تحديد طريقة الافضل ل التنفيذ

Function returns table

طورت SQL Server هذه الميزة، فأصبح التابع قادراً على إعادة table أي row n، وتتم معاملة التابع ك table فيمكن عمل SELECT منه.

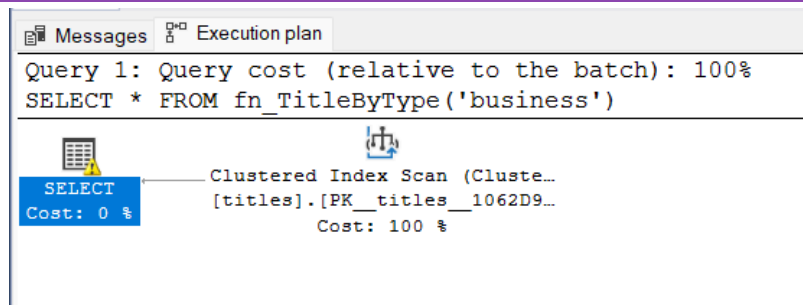
```

CREATE FUNCTION fn_TitleByType(@type NVARCHAR(15))
RETURNS TABLE
AS
RETURN
    (SELECT title_id, title, price, pubdate, pub_id
    FROM titles
    WHERE type = @type);
GO

```

ومع التنفيذ:

```
SELECT * FROM fn_TitleByType('business');
```

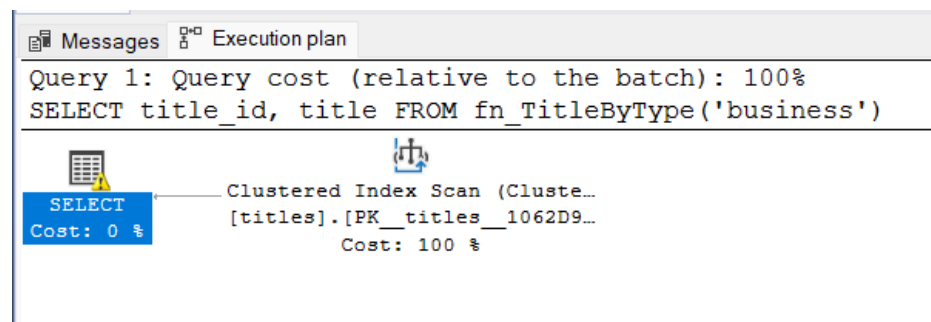


ولنقم بتنفيذ التعليمات التالية لنوضح كيفية استخدام التابع الذي يعيد الـ table:

```

SELECT title_id, title
FROM fn_TitleByType('business');

```



```

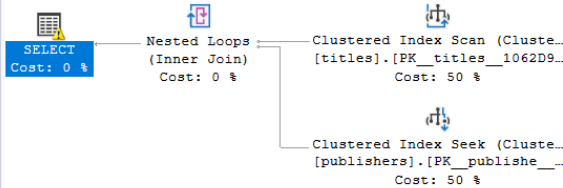
SELECT title, pubdate, pub_name
FROM publishers p
INNER JOIN fn_TitleByType('business') t
ON t.pub_id = p.pub_id;

```

Messages Execution plan

Query 1: Query cost (relative to the batch): 100%

SELECT title, pubdate, pub_name FROM publishers p INNER JOIN fn_TitleByType('business') t ON t.pub_id = p.pub_id



لنقم بتنفيذ:

UPDATE titles

SET price = 100

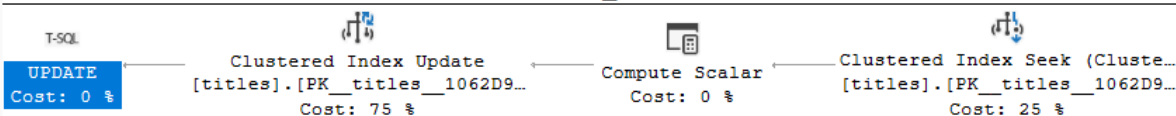
WHERE title_id = 'BU2075'

AND type = 'business';

Messages Execution plan

Query 1: Query cost (relative to the batch): 100%

UPDATE titles SET price = 100 WHERE title_id = 'BU2075' AND type = 'business'



ونلاحظ أنه في جميع الأمثلة السابقة تم معاملة التابع على أنه جدول، فقد تم استدعاؤه في كل منها.

إحدى فوائد التابع المهمة وهي تجنب الـ View:

لنقم ببناء View بالشكل التالي:

CREATE VIEW SOSO

AS

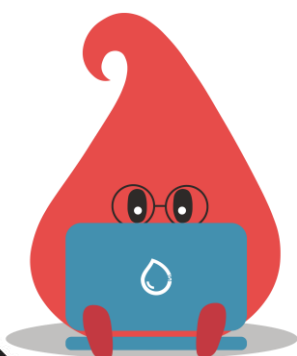
SELECT * FROM titles

SELECT * FROM SOSO;

نلاحظ أنه لا يتم بناء table، وإنما في كل مرة يتم استدعاء SOSO في query ما، سيتم تنفيذ الـ body الخاص بها من جديد. على عكس الـ function الذي يقوم ببناء جدول عند أول تنفيذ فقط، ثم نصبح قادرين على تنفيذ عمليات DML بشكل مباشر على الـ table، بينما الـ View لا يمكنه تنفيذها.

الفرص لا تأتي فجأة

أنت من تصبح جاهزاً فجأة



ملاحظات:

- النتيجة التعليمية: الدالة (Function) تجعل الكود جميلاً ومنظماً، لكن الـ JOIN غالباً ما يكون أسرع في الأداء عند التعامل مع آلاف السجلات.
- غالباً يكون INNER JOIN أفضل من Sub Query من ناحية القراءة والأداء، لكن يعتمد ذلك على Execution Plan.
- ربط مع الـ Optimization: اختيار الطريقة الأنسب يتم بناءً على حجم البيانات والفهارس (Indexes).
- توضيح مصطلح Loop: LOOP تعني تنفيذ استعلام متكرر لكل صف ناتج، مما قد يسبب بطئاً في الأداء. كثرة عمليات Loop تشير غالباً إلى تصميم استعلام غير مثالي.
- تعريف Logical Reads: تمثل عدد الصفحات التي تم قراءتها من الذاكرة. كلما قل عدد الـ Logical Reads كان الأداء أفضل.
- لا توجد قاعدة ثابتة لاختيار أفضل طريقة، وإنما يتم الاعتماد على التجربة والتحليل.
- Database Administrator مسؤول عن تحسين الأداء عبر Tuning و Indexing.
- View: يعيد تنفيذ Query في كل مرة. وهو object من الـ DB يتم بناءه على تعليمة معينة.
- Function: يبني Table مرة واحدة.
- لماذا يعتبر الـ FUNCTION أفضل؟

يعطي مرونة أكبر في التنفيذ ودعم أفضل لـ DML.

- Compile: مرحلة التأكد من صحة الاستعلام نحويًا ومنطقيًا.
- Execution Plan: الخطة التي يضعها محرك قواعد البيانات لتنفيذ الاستعلام بأفضل طريقة.
- Cache Memory: ذاكرة مؤقتة تُخزن فيها خطط التنفيذ لتسريع الاستدعاءات اللاحقة.
- تخزين Execution Plan في Cache Memory لا يعني دائماً أفضل أداء، لأن تغيير حجم البيانات قد يتطلب إعادة توليد الخطة.
- الاستعلام العادي (Query) ينفذ أمراً واحداً في العادة، لكن Stored Procedure يسمح لك بوضع "حزمة" أوامر معاً.
- Function لا يمكن أن تحتوي INSERT / UPDATE / DELETE بينما Procedure يمكنها ذلك.

لماذا؟

لا يمكن استخدام INSERT أو UPDATE أو DELETE داخل Function لأن الدالة تُستخدم ضمن الاستعلامات ويجب أن تكون بدون تأثيرات جانبية، بينما Stored Procedure تُنفذ بشكل مستقل ومصممة لتنفيذ العمليات التعديلية على البيانات.

المقارنة	Function	Procedure
SELECT تُستخدم داخل	✓	✗
تعدّل بيانات	✗	✓
Value ترجع	✓	✓
آمنة ومتوقعة	✓	✗ (حسب الاستخدام)

The End...

إن لم تتحدى نفسك
ستعيش نسخة ضعيفة منك

